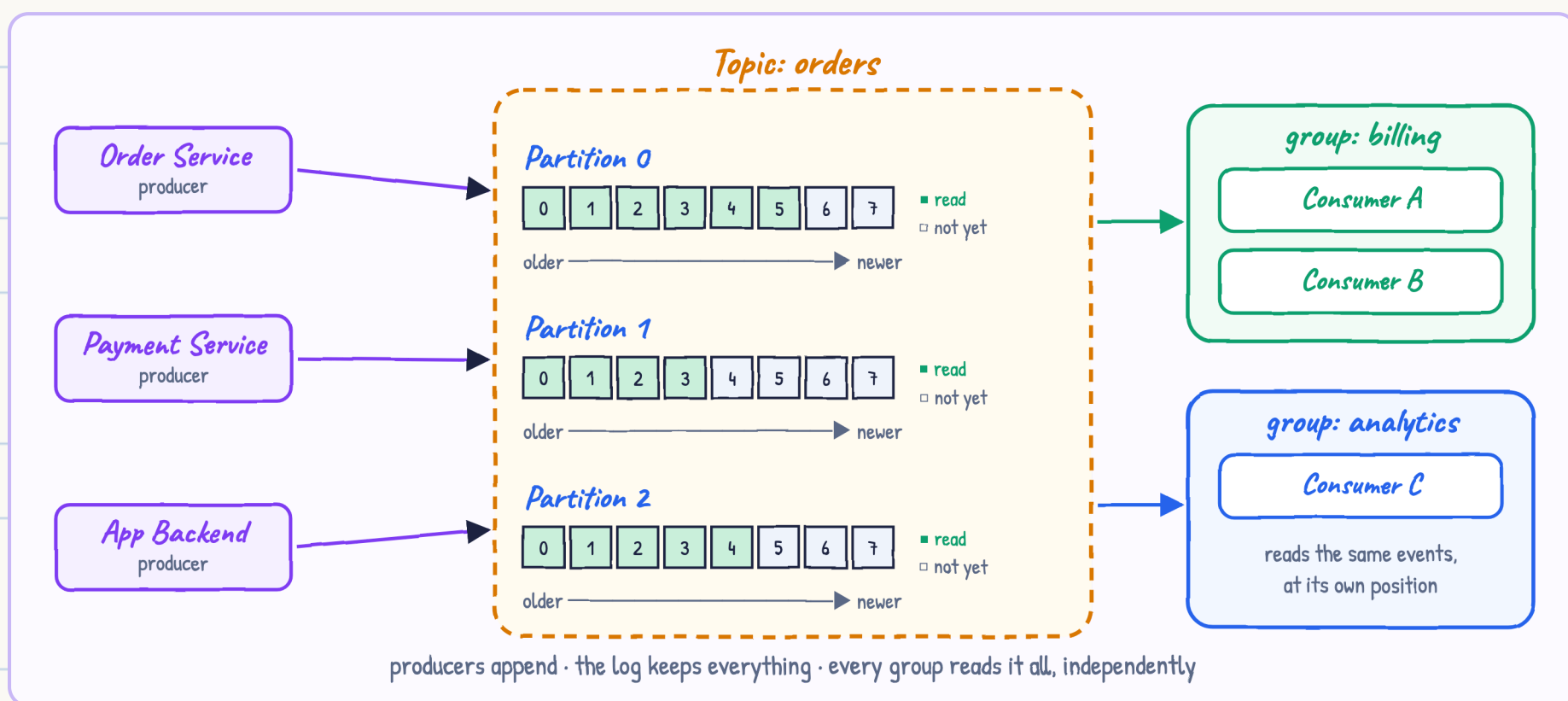




Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed



What's inside

- ✓ What Kafka actually is
- ✓ Topic, Partition, Offset
- ✓ The Producer
- ✓ Consumer & Consumer Groups
- ✓ Brokers & the Cluster
- ✓ Ordering – the most misunderstood part
- ✓ Delivery: what can go wrong
- ✓ Retention & compaction
- ✓ Kafka vs a normal queue
- ✓ Where Kafka is really used
- ✓ Interview questions

Who it's for

- ✓ Developers who keep hearing "Kafka" and nodding along
- ✓ Backend engineers preparing for interviews
- ✓ Anyone who has used Kafka without understanding it
- ✓ Students who want the mental model, not commands

Why it matters

- ✓ Every idea explained with a picture
- ✓ Plain analogies before any jargon
- ✓ Built up in order – each page uses the last
- ✓ Ends with interview answers you can say out loud

Comment KAFKA and I will DM you the full PDF



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

1

What is Kafka?



Kafka

Apache Kafka is a **distributed event streaming platform**. Services publish events to it, and other services read those events – **without knowing about each other**.

- An **event** is a fact that happened: "order 91 was paid"
- Kafka stores events in order, on disk, for a period of time
- Any number of services can read the same events
- Producers and consumers never talk directly



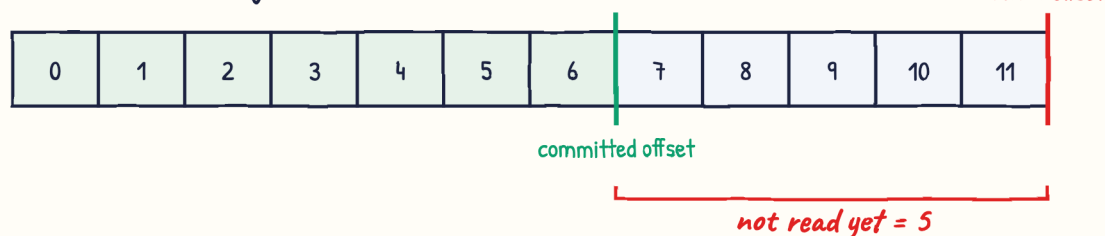
Simple analogy

- Think of a **newspaper**, not a phone call
- The paper is printed once and **many people** read it
- A reader has a **bookmark** – how far they have read
- The publisher does not know or wait for any reader
- Yesterday's paper is still on the shelf for a while

Why do we need it at all?

Producers write, consumers read – independently

Partition 0 – the log



the log is written once · each reader keeps its own position

1

Decoupling



- The order service just publishes "order paid"
- Billing, email and analytics each react alone
- Adding a 4th consumer needs **no change** to the producer

2

Buffering



- A slow consumer does not slow the producer
- Events wait until the consumer catches up
- Traffic spikes are absorbed instead of dropped

3

Replay



- Move your bookmark back and read again
- Rebuild a cache or a report from history
- A phone call cannot be replayed – a log can

4

One source of truth



- Every service sees the **same** events in the same order
- No service has to ask another for missing data
- The log is the shared record of what happened



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

2

Topic, Partition, Offset

The three words everything else is built on

1

Topic



- A named category of events – `orders`, `payments`
- Like a folder, or a newspaper section
- Purely a label; the real data sits in partitions

2

Partition



- A topic is `split` into partitions
- Each partition is one ordered list of events
- Splitting is what lets many consumers work at once

3

Offset



- The position of an event inside a partition
- Event 0, event 1, event 2 ... it only grows
- A consumer remembers "I have read up to 7"

4

Why split at all



- One list can only be read by one reader at a time
- Three partitions → three readers in parallel
- Throughput scales with partition count

5

What splitting costs



- Order is kept `inside` a partition, not across them
- Events for the same user must go to the same partition
- That is why producers choose a partition by `key`

6

Putting it together



- Topic = what kind of event
- Partition = which ordered list it lives in
- Offset = where in that list it sits

Key

- ✓ A topic is a name · a partition is an ordered list · an offset is a position in that list

Takeaways

- ✓ Ordering exists `inside a partition` – never across a whole topic
- ✓ Same key → same partition → events for that entity stay in order



Kafka Complete Handbook

Understand how Kafka works – from scratch

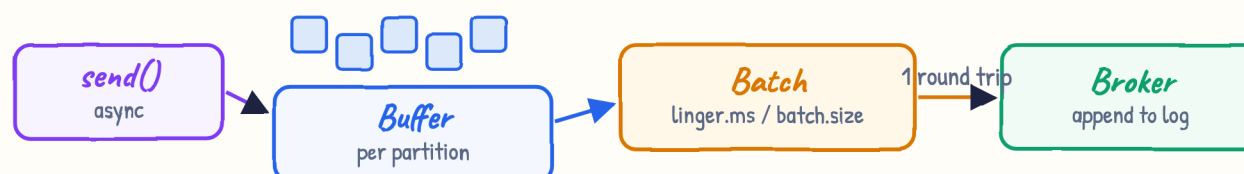
Concepts, analogies and diagrams – no prior experience needed

3

The Producer

How an event gets written

From your code to the log



thousands of records travel in one network round trip – that is the speed

the producer **groups** events before sending – that is where the speed comes from



Choosing a partition

- If you give a **key**, Kafka hashes it → same key, same partition
- Key by `user_id` or `order_id` when order matters per entity
- No key → events are spread evenly, **order not guaranteed**
- This one choice decides both ordering and load balance



Why batching matters

- Sending one event per network call would be very slow
- The producer waits a few milliseconds and sends **a batch**
- One round trip can carry thousands of events
- This is why Kafka handles volumes a queue cannot

1

Fire and forget



- Send and do not wait for confirmation
- Fastest, but you never learn if it failed
- Fine for metrics, wrong for payments

2

Wait for the leader



- The broker confirms it wrote the event
- Fast, but a crash before copying loses it
- A middle ground, rarely the right one

3

Wait for the copies



- Confirm only once replicas also have it
- Slower, and the only truly safe option
- This is what you want for anything that matters



Kafka Complete Handbook

Understand how Kafka works – from scratch

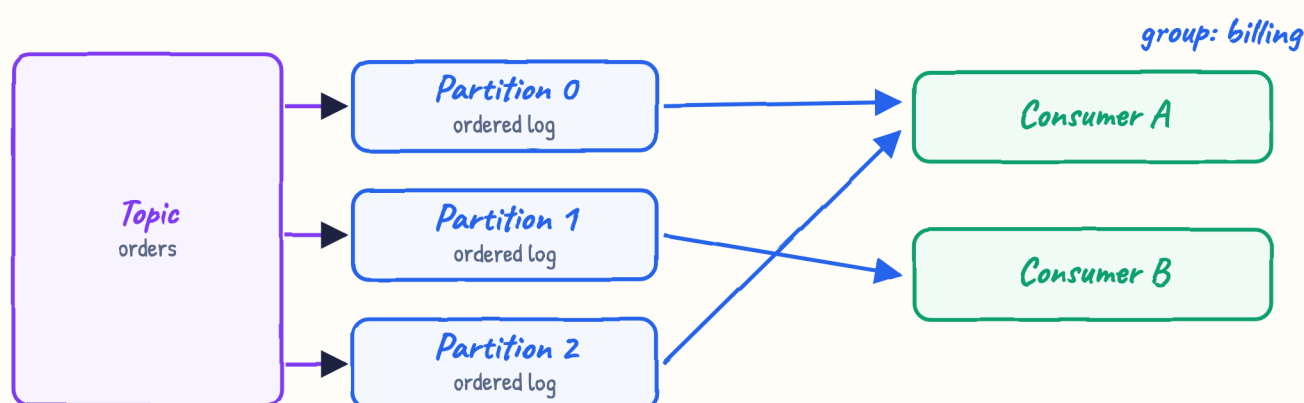
Concepts, analogies and diagrams – no prior experience needed

4

The Consumer & Consumer Groups

How reading actually works

Three partitions shared between two consumers



1

A consumer group



- Several copies of the same service working together
- Kafka **divides** the partitions between them
- If one dies, its partitions move to the others

2

One owner per partition



- A partition is read by exactly one member of a group
- That is how Kafka avoids locks completely
- So members beyond the partition count sit **idle**

3

Two different groups



- Billing and analytics are separate groups
- Each group receives **every** event
- One being slow does not affect the other

4

The bookmark



- A group stores how far it has read, per partition
- Restart and it continues from there
- Move the bookmark back and it reads history again

Key

Takeaways

- ✓ Group = a team splitting the work · a new group = read everything from the start
- ✓ Adding consumers past the partition count adds **nothing**
- ✓ The bookmark belongs to the group, not to the individual consumer



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

5

Brokers & the Cluster

Where the data physically lives



Broker

A broker is one Kafka server. A cluster is several brokers that split the partitions between them.

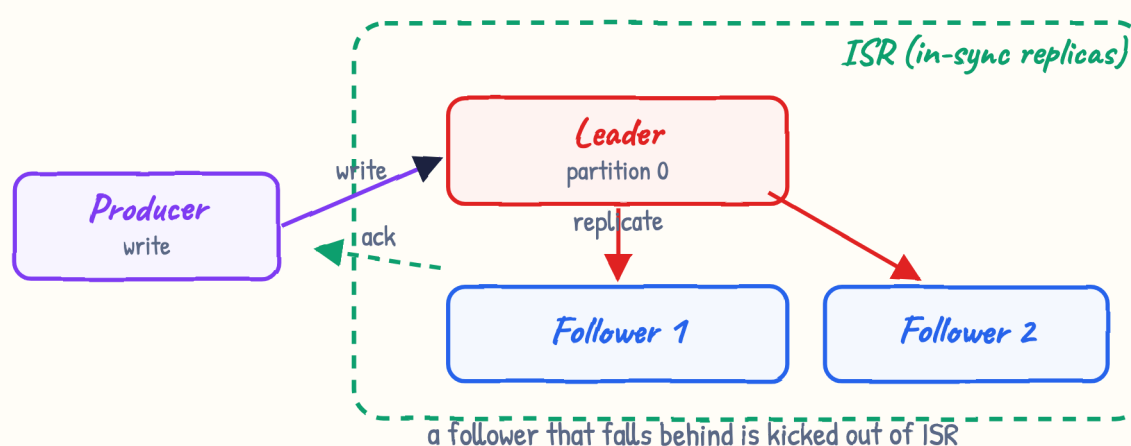
- Each partition is stored as files on one broker's disk
- That broker is the leader for the partition
- Other brokers keep a copy – they are followers
- Clients always read and write through the leader



Why more than one

- One machine would be a single point of failure
- One machine's disk and network would cap throughput
- Partitions spread across brokers spread the load
- Copies across brokers survive a machine dying

Leader and its copies



the leader takes the write · followers copy it · a follower can take over

1

If a broker dies



- One of its followers becomes the new leader
- Clients reconnect to the new leader automatically
- With copies in place, nothing is lost

2

Why copies must keep up



- A copy that falls behind is no longer trustworthy
- Kafka stops counting it as a valid backup
- Promoting a stale copy would lose events



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

6

Ordering – the most misunderstood part

What Kafka does and does not promise



The promise

Kafka guarantees order **within one partition**. It makes **no promise** about order across a topic.

- Inside a partition, events are read in the order written
- Across partitions, two events can arrive in any order
- This is not a limitation – it is the price of parallelism



Why it works that way

- Global ordering would need one single list
- One list means one reader – no parallelism at all
- Kafka trades global order for **scale**
- You choose where order matters, using the key

1

Order per user



- Key by `user_id`
- All of that user's events land in one partition
- Their sequence is preserved perfectly

2

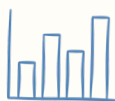
Order per order



- Key by `order_id`
- `created` → `paid` → `shipped` stays in sequence
- Different orders may interleave – and that is fine

3

No order needed



- Page views, metrics, logs
- Leave the key empty for an even spread
- Maximum throughput

4

The classic mistake



- Expecting the whole topic to be in order
- It never is, and no setting makes it so
- Design around per-entity ordering instead



Kafka Complete Handbook

Understand how Kafka works – from scratch

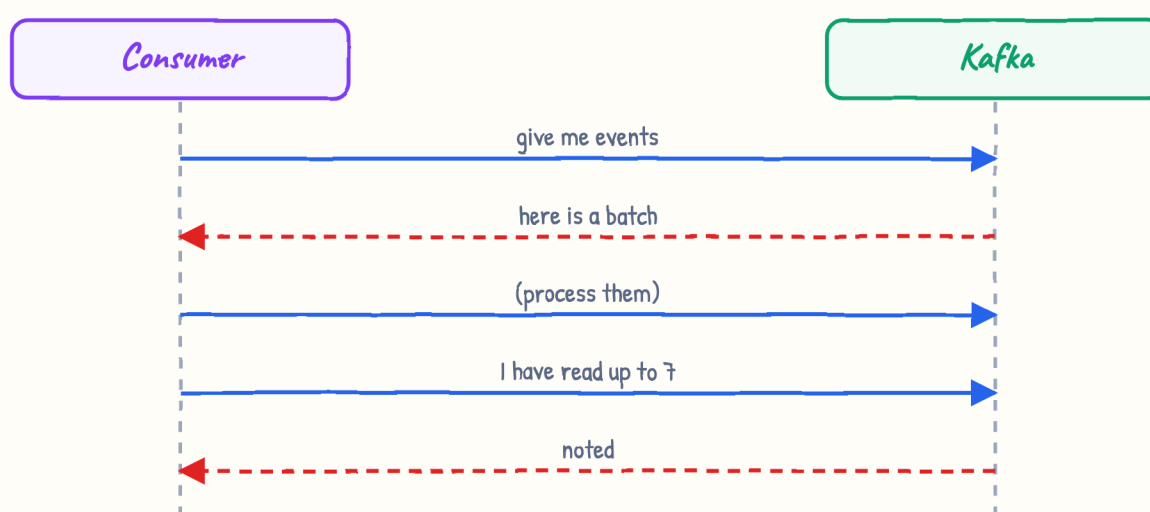
Concepts, analogies and diagrams – no prior experience needed

7

Delivery – what can go wrong

Why duplicates exist, and what to do about them

Read, process, then remember



if it crashes **before** the last step, it re-reads from 5

1

Remember first



- Mark it read, then process
- If you crash mid-work, the event is **gone**
- Called at-most-once – you can lose data

2

Remember after



- Process, then mark it read
- A crash means you **process it twice**
- Called at-least-once – the normal choice

3

Handle the double



- Make the work safe to repeat
- Same input, same result, no side effect twice
- This is what people mean by **idempotent**

4

How in practice



- Save the event id and skip it if already seen
- Or write with an **UPSERT** instead of **INSERT**
- The safety lives in **your code**, not in Kafka

Key

- ✓ There is no setting that gives you exactly-once for free

Takeaways

- ✓ Pick at-least-once, then make your consumer **safe to repeat**
- ✓ A duplicate that changes nothing is not a bug



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

8

Retention – when events disappear

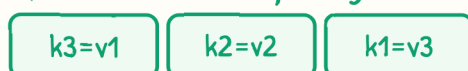
Kafka is not a database. So how long does it keep things?

Compaction: keep the latest value for each key

before compaction



after – latest value per key



nothing is deleted when you read – only when it gets **old**, or is replaced



Delete by age

The normal mode. Kafka keeps events for a **time window**, then removes the oldest files.

- Reading has no effect on this – it is a clock
- An event nobody read still expires
- Perfect for event streams: orders, clicks, logs



Keep the latest

Compaction keeps only the **most recent** event per key and drops older versions of that key.

- The topic becomes a snapshot of current state
- Replay it to rebuild a full lookup table
- Perfect for: user profiles, prices, settings

1

The trap



- "Unread means safe" is **wrong**
- A very slow consumer can be overtaken by deletion
- Watch how far behind your consumers are

2

Deleting one key



- In a compacted topic, write the key with an empty value
- That marker tells Kafka to drop the key
- There is no DELETE statement in Kafka



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

9

Kafka or a normal queue?

They look similar and solve different problems



The real difference

	Queue (RabbitMQ, SQS)	Kafka (log)
When you read	the message is removed	your bookmark moves
Multiple readers	they compete for messages	each group gets everything
Reading again	not possible	move the bookmark back
Ordering	per queue	per partition
Retry one message	yes – built in	no, you handle it
Best for	tasks and jobs	events and streams

1

Choose Kafka when



- Several services need the same events
- You may want to replay or audit later
- The event history itself has value
- Volume is high and sustained

2

Choose a queue when



- One worker should pick up one job
- You need retry, priority or delay per message
- Volume is modest
- You do not want to run a cluster



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

10

Where Kafka is actually used

Real systems you already use

1

Ride hailing



- Driver location pings, thousands per second
- One stream feeds pricing, ETA and the map
- Replay lets them rebuild a trip after the fact

2

Payments



- Every state change published as an event
- Ledger, notifications and fraud all read it
- The log becomes the audit trail

3

E-commerce



- "Order placed" fans out to a dozen services
- Inventory, invoicing, email, warehouse, analytics
- None of them knows the others exist

4

Streaming media



- Every play, pause and seek becomes an event
- Feeds recommendations and billing
- Volumes far beyond what a queue handles

5

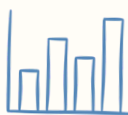
Banking



- Transaction events replicated across systems
- Core banking, reporting and compliance
- Replay is a regulatory requirement

6

Logs & metrics



- Every service ships events to one place
- Search, dashboards and alerting read the same stream
- No key needed – pure throughput



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

11

Interview Questions

The answers that land

1

What is Kafka in one line?



- "A distributed log. Producers append events, consumers read at their own position, and reading does not remove anything."

2

Queue vs Kafka?



- "A queue deletes on read so only one consumer gets a message. Kafka keeps the event, so every consumer group reads all of it and can replay."

3

How is ordering guaranteed?



- "Only within a partition. Keying by entity id puts all of that entity's events in one partition, which is where ordering comes from."

4

Why did more consumers not help?



consumer per group, so once you match the partition count, extra consumers are idle."

5

How do you avoid duplicates?



- "You do not – you make the consumer idempotent. Dedupe on an event id, or write with an upsert so repeating is harmless."

6

Is Kafka a database?



- "No. It keeps events for a retention window, not forever, and you read it by position – not by query."



Kafka Complete Handbook

Understand how Kafka works – from scratch

Concepts, analogies and diagrams – no prior experience needed

That's Kafka, end to end.

What you just went through

- | | |
|---|--|
| 1 What is <i>Kafka</i> ? | 7 Delivery – what can go wrong |
| 2 Topic, <i>Partition</i> , Offset | 8 Retention – when events disappear |
| 3 The <i>Producer</i> | 9 <i>Kafka</i> or a normal queue? |
| 4 The <i>Consumer</i> & Consumer Groups | 10 Where <i>Kafka</i> is actually used |
| 5 <i>Brokers</i> & the Cluster | 11 Interview Questions |
| 6 <i>Ordering</i> – the most misunderstood part | |



Save this – you will come back to it



Share it with someone still nodding along



Comment **KAFKA** for the complete PDF in your DM

Written from 10 years of backend engineering – Java, Kafka, Redis, Postgres in production.

@vikstack